

[linear.algebra]R2

A proposal to add linear algebra support to the C++ standard library

Working Draft, 2019-06-19

This version:

https://github.com/BobSteagall/wg21/blob/master/linear_algebra/papers/P1385R2.html

Latest published version:

<http://wg21.link/P1385>

Authors:

[Guy Davidson](#)

[Bob Steagall](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

SG14, SG6, LEWG

Previous version:

<http://wg21.link/P1385R1>

Strawman implementation:

https://github.com/BobSteagall/wg21/tree/master/linear_algebra/code

Abstract

This document proposes a set of fundamental linear algebra types and functions for the standard C++ library.

Table of Contents

1 Introduction

2 Goals

3 Definitions

3.1 Mathematical terms

3.2 Terms pertaining to C++ types

3.3 Overloaded terms

3.3.1 Matrix

3.3.2 Vector

3.3.3 Dimension

3.3.4 Rank

4 Scope

4.1 Functional requirements

4.2 Considered but excluded

5 Design aspects

5.1 Memory source

5.2 Addressing model

5.3 Memory ownership

5.4 Capacity and resizing

5.5 Element layout

5.6 Element access and indexing

5.7 Element type

5.8 Mixed-element-type expressions

5.9 Mixed-engine expressions

5.10 Arithmetic customization

5.11 Linear algebra and `constexpr`

6 Interface description

6.1 Overview

6.2 `std::math` namespace

6.3 Header `<linear_algebra>` synopsis

6.4 Numeric traits

6.4.1 `number_traits<T>`

6.4.2 `is_complex<T>`

6.4.3 `is_field<T>`

6.4.4 `is_ring<T>`

6.4.5 `is_nc_ring<T>`

6.4.6 `is_matrix_element<T>`

6.5 Engine Types

6.5.1 `matrix_column_view<ET>`

6.5.2 `matrix_row_view<ET>`

6.5.3 `matrix_transpose_view<ET>`

6.5.4 `fs_vector_engine<T, N>`

6.5.5 `fs_matrix_engine<T, R, C>`

6.5.6 `dr_vector_engine<T, AT>`

6.5.7 `dr_matrix_engine<T, AT>`

6.6 Math object types

6.6.1 `vector<ET, OT>`

6.6.2 `matrix<ET, OT>`

6.7 Operation traits

6.7.1 `matrix_operation_traits`

6.7.2 `matrix_operation_traits_selector<OT1, OT2>`

6.8 Element promotion traits

6.8.1 `matrix_negation_element_traits<T1>`

6.8.2 `matrix_addition_element_traits<T1, T2>`

6.8.3 `matrix_subtraction_element_traits<T1, T2>`

6.8.4 `matrix_multiplication_element_traits<T1, T2>`

6.9 Engine promotion traits

6.9.1 `matrix_negation_engine_traits<OT, ET1>`

6.9.2 `matrix_addition_engine_traits<OT, ET1, ET2>`

6.9.3 `matrix_subtraction_engine_traits<OT, ET1, ET2>`

6.9.4 `matrix_multiplication_engine_traits<OT, ET1, ET2>`

6.10 Arithmetic traits

6.10.1 `matrix_negation_traits<OT, OP1>`

6.10.2 `matrix_addition_traits<OT, OP1, OP2>`

6.10.3 `matrix_subtraction_traits<OT, OP1, OP2>`

6.10.4 `matrix_multiplication_traits<OT, OP1, OP2>`

6.11 Arithmetic operators

7 Customization

- 7.1 Integrating a new element type
- 7.2 Custom element promotion
- 7.3 Integrating a new engine type
- 7.4 Customizing an arithmetic operation

8 Meeting feedback

- 8.1 Kona
 - 8.1.1 LEWG(I) Polls and Feedback (Wednesday 2019-02-20)
 - 8.1.2 Feedback From Joint SG14/SG19 Session (Friday 2019-02-22)
 - 8.1.3 Other Suggestions Gathered at the Meeting

Revision History

Version	Description
R0	Initial version for pre-Kona mailing
D1	Update for presentation at Kona; includes operation traits.
R1	Update for post-Kona mailing; includes feedback from LEWG(I) and joint SG14/SG19 session.
R2	Update for Cologne meeting; includes feedback from Kona and monthly conference calls. - Emphasized proposed <code>std::math</code> namespace - Replaced <code>row_vector</code> and <code>column_vector</code> types with a single <code>vector</code> type to represent both. - Removed discussion regarding 0-based or 1-based indexing in favor of 0-based. - Reduced number of customization points within namespace <code>std</code> to two.

Open issues

1. Explore how to specify concepts for numerical types.
2. Explore use of boolean `true/false` vs. `std::true_type/std::false_type` for nested property predicates in math types.
3. Explore use of `mdspan` as a component for implementation.

4. Explore interface designs to support distributed/parallel/multithreaded arithmetic.
5. Explore value-based and expression-based approaches to the arithmetic operators.
6. Re-visit one-vector vs. two-vector approach.
7. Add type requirements tables.
8. Add a Gor table.
9. Develop tutorial materials and examples.

§ 1. Introduction

Linear algebra is a mathematical discipline of ever-increasing importance, with direct application to a wide variety of problem domains, such as signal processing, computer graphics, medical imaging, scientific simulations, machine learning, analytics, financial modeling, and high-performance computing. And yet, despite the relevance of linear algebra to so many aspects of modern computing, the C++ standard library does not include a set of linear algebra facilities. This paper proposes to remedy this deficit for C++23.

This paper should be read after P1166, in which we describe a high-level set of expectations for what a linear algebra library should contain.

§ 2. Goals

We expect that typical users of a standard linear algebra library are likely to value two features above all else: ease-of-use (including expressiveness), and high performance out of the box. This set of users will expect the ability to compose arithmetical expressions of linear algebra objects similar to what one might find in a textbook; indeed, this has been deemed a "must-have" feature by several participants in recent SG14 Linear Algebra SIG conference calls. And for a given arithmetical expression, they will expect run-time computational performance that is close to what they could obtain with an equivalent sequence of function calls to a more "traditional" linear algebra library, such as *LAPCK*, *Blaze*, *Eigen*, etc.

There also exists a set of linear algebra “super-users” who will value most highly a third feature – the ability to customize underlying infrastructure in order to maximize performance for specific problems and computing platforms. These users seek the highest possible run-time performance, and to achieve it, require the ability to customize any and every portion of the library’s computational infrastructure.

With these high-level user requirements in mind, in this paper we propose an interface specification intended to achieve the following goals:

1. To provide a set of vocabulary types for representing the mathematical objects and operations that are relevant to linear algebra;
2. To provide a public interface for linear algebra operations that is intuitive, teachable, and mimics the expressiveness of mathematical notation to the greatest *reasonable* extent;
3. To exhibit out-of-the-box performance in the neighborhood of that of that exhibited by an equivalent sequence of function calls to a more traditional linear algebra library, such as *LAPACK*, *Blaze*, *Eigen*, etc.;
4. To provide a set of building blocks that manage the source, ownership, lifetime, layout, and access of the memory required to represent the linear algebra vocabulary types, with the requirement that some of these building blocks are also suitable for (eventually) representing other interesting mathematical entities, such as quaternions, octonions, and tensors;
5. To provide straightforward facilities and techniques for customization that enable users to optimize performance for their specific problem domain on their specific hardware; and,
6. To provide a *reasonable* level of granularity for customization so that developers only have to implement a minimum set of types and functions to integrate their performance enhancements with the rest of the linear algebra facilities described here.

§ 3. Definitions

When discussing linear algebra and related topics for a proposal such as this, it is important to note that there are several overloaded terms (such as *matrix*, *vector*, *dimension*, and *rank*) which must be defined and disambiguated if such discussions are to be productive. These terms have specific meanings in mathematics, as well as different, but confusingly similar, meanings to C++ programmers.

In the following sections we provide definitions for relevant mathematical concepts, C++ type design concepts, and describe how this proposal employs those overloaded terms in various contexts.

§ 3.1. Mathematical terms

In order to facilitate subsequent discussion, we first provide the following *informal* set of definitions for important mathematical concepts:

1. A **vector space** is a collection of **vectors**, where vectors are objects that may be added together and multiplied by scalars. Euclidean vectors are an example of a vector space, typically used to

represent displacements, as well as physical quantities such as force or momentum. Linear algebra is concerned primarily with the study of vector spaces.

2. The **dimension** of a vector space is the minimum number of coordinates required to specify any point within the space.
3. A **matrix** is a rectangular array of numbers, symbols, or expressions, arranged in rows and columns. A matrix having m rows and n columns is said to have size $m \times n$. Although matrices can be used solve systems of simultaneous linear equations, they are most commonly used to represent linear transformations, solve linear least squares problems, and to explore and/or manipulate the properties of vector spaces.
4. The **rank** of a matrix is the dimension of the vector space spanned by its columns, which is equal to the dimension of the vector space spanned by its rows. The rank is also equal to the maximum number of linearly-independent columns and rows.
5. An **element** of a matrix is an individual member (number, symbol, expression) of the rectangular array comprising the matrix, lying at the intersection of a single row and a single column. In traditional mathematical notation, row and column indexing is 1-based, where rows are indexed from 1 to m and columns are indexed from 1 to n . Given some matrix A , element a_{11} refers to the element in the upper left-hand corner of the array and element a_{mn} refers to the element in the lower right-hand corner.
6. A **row vector** is a matrix containing a single row; in other words, a matrix of size $1 \times n$. In many applications of linear algebra, row vectors represent spatial vectors.
7. A **column vector** is a matrix containing a single column; in other words, a matrix of size $m \times 1$. In many applications of linear algebra, column vectors represent spatial vectors.
8. **Element transforms** are non-arithmetical operations that modify the relative positions of elements in a matrix, such as transpose, column exchange, and row exchange.
9. **Element arithmetic** refers to arithmetical operations that read or modify the values of individual elements independently of other elements, such as assigning a value to a specific element or multiplying a row by some value.
10. **Matrix arithmetic** refers to the assignment, addition, subtraction, negation, multiplication, and determinant operations defined for matrices, row vectors, and column vectors as wholes.
11. A **rectangular matrix** is a matrix requiring a full $m \times n$ representation; that is, a matrix not possessing a special form, such as identity, triangular, band, etc.
12. The **identity matrix** is a square matrix where all elements on the diagonal are equal to one and all off-diagonal elements are equal to zero.
13. A **triangular matrix** is a matrix where all elements above or below the diagonal are zero; those with non-zero elements above the diagonal are called *upper triangular*, while those with non-zero

elements below the diagonal are called *lower triangular*.

14. A **band matrix** is a sparse matrix whose non-zero entries are confined to a diagonal band, lying on the main diagonal and zero or more diagonals on either side.
15. **Decompositions** are complex sequences of arithmetic operations, element arithmetic, and element transforms performed upon a matrix that expose important mathematical properties of that matrix. Several types of decomposition are often performed in solving least-squares problems.
16. **Eigen-decompositions** are decompositions performed upon a symmetric matrix in order to compute the eigenvalues and eigenvectors of that matrix; this is often performed when solving problems involving linear dynamic systems.

§ 3.2. Terms pertaining to C++ types

The following are terms used in this proposal that describe various aspects of how the mathematical concepts described above in Section 3.1 might be implemented:

1. An **array** is a data structure representing an indexable collection of objects (elements) such that each element is identified by at least one index. An array is said to be *one-dimensional* array if its elements are accessible with a single index; a *multi-dimensional* array is an array for which more than one index is required to access its elements.
2. The **dimension** of an array refers to the number of indices required to access an element of that array. The **rank** of an array is a synonym for its dimension.
3. This proposal uses the term **MathObj** to refer generically to one of the C++ types described herein representing matrices and vectors (i.e., **matrix** and **vector**). These are the public-facing types developers will use in their code.
4. An **engine** is an implementation type that manages the resources associated with a *MathObj* instance. This includes, at a minimum, the storage-related aspects of, and access to, the elements of a *MathObj*. It could also include execution-related aspects, such as an execution context. In this proposal, an engine object is a private member of a *MathObj*. Other than as a template parameter, engines are not part of a *MathObj*'s public interface.
5. The adjective **dense** refers to a *MathObj* representation where storage is allocated for every element.
6. The adjective **sparse** refers to a *MathObj* representation where storage is allocated only for non-zero elements;
7. **Storage** is used by this proposal as a synonym for memory.

8. **Traits** refers to a stateless class template that provides some set of services, normalizing those services over its set of template parameters.
9. **Row size** and **column size** refer to the number of rows and columns, respectively, that a *MathObj* represents, which must be less than or equal to its row and column capacities, defined below.
10. **Row capacity** and **column capacity** refer to the maximum number of rows and columns, respectively, that a *MathObj* can possibly represent.
11. **Fixed-size** (FS) refers to an engine type whose row and column sizes are fixed at instantiation time and constant thereafter.
12. **Fixed-capacity** (FC) refers to an engine type whose row and column capacities are fixed at instantiation time and constant thereafter.
13. **Dynamically re-sizable** (DR) refers to an engine type whose row and column sizes and capacities may be changed at run time.

§ 3.3. Overloaded terms

This section describes how we use certain overloaded terms in this proposal and in future works.

§ 3.3.1. Matrix

The term *matrix* is frequently used by C++ programmers to mean a general-purpose array of arbitrary size. For example, one of the authors worked at a company where it was common practice to refer to 4-dimensional arrays as “4-dimensional matrices.”

In this proposal, we use the word *array* only to mean a data structure whose elements are accessible using one or more indices, and which has no invariants pertaining to higher-level or mathematical meaning.

We use *matrix* to mean the mathematical object as defined above in Section 3.1, and `matrix` (in monospaced font) to mean the C++ class template that implements the mathematical object. We sometimes use `MathObj` (in monospaced font) in some of the component interface code and text below to generically refer to a `matrix`, `row_vector`, or `column_vector` object.

§ 3.3.2. Vector

Likewise, many C++ programmers incorrectly use the term *vector* as a synonym for “dynamically re-sizable array.” This bad habit is exacerbated by the unfortunate naming of `std::vector`.

This proposal uses the term *vector* to mean an element of a vector space, per Section 3.1 above. Further, *row vector* and *column vector* have the meanings set out in 3.1, while `row_vector` and `column_vector` (in monospaced font) are the C++ class templates implementing those mathematical objects, respectively. We sometimes use `MathObj` (in monospaced font) in some of the component code interface code and test below to generically refer to a `row_vector`, `column_vector`, or `matrix` object.

§ 3.3.3. Dimension

In linear algebra, a vector space V is said to be of *dimension* n , or be *n-dimensional*, if there exist n linearly independent vectors which span V . This is another way of saying that n is the minimum number of coordinates required to specify any point in V . However, in common programming parlance, *dimension* refers to the number of indices used to access an element in an array.

We use the term dimension both ways in this proposal, but try to do so consistently and in a way that is clear from the context. For example, a rotation matrix used by a game engine is two-dimensional data structure composed of three-dimensional row and column vectors. A vector describing an electric field is an example of a one-dimensional data structure that implemented as a three-dimensional column vector.

§ 3.3.4. Rank

The *rank* of a matrix is the dimension of the vector space spanned by its columns (or rows), which corresponds to the maximal number of linearly independent columns (or rows) of that matrix. Rank also has another meaning in tensor analysis, where it is commonly used as a synonym for a tensor's *order*.

However, rank also has a meaning in computer science where it is used as a synonym for dimension. In the C++ standard at `[meta.unary.prop.query]`, rank is described as the number of dimensions of `T` if `T` names an array, otherwise it is zero.

We avoid using the term *rank* in this proposal in the context of linear algebra, except as a quantity that might result from performing certain decompositions wherein the mathematical rank of a matrix is computed.

§ 4. Scope

We contend that the best approach for standardizing a set of linear algebra components for C++23 will be one that is layered, iterative, and incremental. This paper is quite deliberately a “basic linear algebra-only” proposal; it describes what we believe is a foundational layer providing the minimum set of components and arithmetic operations necessary to provide a reasonable, basic level of functionality.

Higher-level functionality can be specified in terms of the interfaces described here, and we encourage succession papers to explore this possibility.

§ 4.1. Functional requirements

The foundational layer, as described here, should include the minimal set of types and functions required to perform matrix arithmetic in finite dimensional spaces. This includes:

- Matrix, row vector, and column vector class templates;
- Arithmetic operations for addition, subtraction, negation, and multiplication of matrices and row/column vectors;
- Arithmetic operations for scalar multiplication of matrices and row/column vectors;
- Well-defined facilities for integrating new element types;
- Well-defined facilities for creating and integrating custom engines; and,
- Well-defined facilities for creating and integrating custom arithmetic operations.

§ 4.2. Considered but excluded

Tensors

There has been a great deal of interest expressed in specifying an interface for general-purpose tensor processing in which linear algebra facilities fall out as a special case. We exclude this idea from this proposal for two reasons. First, given the practical realities of standardization work, the enormous scope of such an effort would very likely delay introduction of linear algebra facilities until C++26 or later.

Second, and more importantly, implementing matrices as derived types or specializations of a general-purpose tensor type is bad type design. Consider the following: a tensor is (informally) an array of mathematical objects (numbers or functions) such that its elements transform according to certain rules

under a coordinate system change. In a p -dimensional space, a tensor of rank n will have p^n elements. In particular, a rank-2 tensor in a p -dimensional space may be represented by a $p \times p$ matrix having certain invariants related to coordinate transformation not possessed by all $p \times p$ matrices.

These defining characteristics of a tensor lead us to the crux of the issue: every rank-2 tensor can be represented by a square matrix, but not every square matrix represents a tensor. As one quickly realizes, only a small fraction of all possible matrices are representations of rank-2 tensors.

All of this is a long way of saying that the class invariants governing a matrix type are quite different from those governing a tensor type, and as such, the public interfaces of such types will also differ substantially.

From this we conclude that matrices are not Liskov-substitutable for rank-2 tensors, and therefore as matter of good type design, matrices and tensors should be implemented as distinct types, perhaps with appropriate inter-conversion operations.

This situation is analogous to the age-old object-oriented design question: when designing a group of classes that represent geometric shapes, is a square a kind of rectangle? In other words, should class `square` be publicly derived from class `rectangle`? Mathematically, yes, a square *is* a rectangle. But from the perspective of good interface design, class `square` is not substitutable for class `rectangle` and is usually best implemented as a distinct type having no IS-A relationship with `rectangle`.

Quaternions and octonions

There has also been interest expressed in including other useful mathematical objects, such as quaternions and octonions, as part of a standard linear algebra library. Although element storage for these types might be implemented using the engines described in this proposal, quaternions and octonions represent mathematical concepts that are fundamentally different from those of matrices and vectors.

As with tensors, the class invariants and public interfaces for quaternions and octonions would be substantially different from that of the linear algebra components. Liskov substitutability would not be possible, and therefore quaternions and octonions should be implemented as a set of types distinct from the linear algebra types.

§ 5. Design aspects

The following describe several important aspects of the problem domain affecting the design of the proposed interface. Importantly, these aspects are orthogonal, and are addressable through judicious combinations of template parameters and implementation type design.

§ 5.1. Memory source

Perhaps the first question to be answered is that of the source of memory in which elements will reside. One can easily imagine multiple sources of memory:

- Elements reside in an external buffer allocated from the global heap.
- Elements reside in an external buffer allocated by a custom allocator and/or specialized heap.
- Elements reside in an external fixed-size buffer that exists independently of the *MathObj*, not allocated from a heap, and which has a lifetime greater than that of the *MathObj*.
- Elements reside in a fixed-size buffer that is a member of the *MathObj* itself.
- Elements reside collectively in a set of buffers distributed across multiple machines.

§ 5.2. Addressing model

It is also possible that the memory used by a *MathObj* might be addressed using what the standard calls a *pointer-like type*, also known as a *fancy pointer*.

For example, consider an element buffer existing in a shared memory segment managed by a custom allocator. In this case, the allocator might employ a fancy pointer type that performs location-independent addressing based on a segment index and an offset into that segment.

One can also imagine a fancy pointer that is a handle to a memory resource existing somewhere on a network, and addressing operations require first mapping that resource into the local address space, perhaps by copying over the network or by some magic sequence of RPC invocations.

§ 5.3. Memory ownership

The next important questions pertain to memory ownership. Should the memory in which elements reside be deallocated, and if so, what object is responsible for performing the deallocation?

A *MathObj* might own the memory in which it stores its elements, or it might employ some non-owning view type, like `mdspan`, to manipulate elements owned by some other object.

§ 5.4. Capacity and resizability

As with `std::string` and `std::vector`, it is occasionally useful for a *MathObj* to have excess storage capacity in order to reduce the number of re-allocations required by anticipated future resizing

operations. Some linear algebra libraries, like LAPACK, account for the fact that a *MathObj*'s capacity may be different than its size. This capability was of critical importance to the success of one author's prior work in functional MRI image analysis.

In other problem domains, like computer graphics, *MathObjs* are small and always of the same size. In this case, the size and capacity are equal, and there is no need for a *MathObj* to maintain or manage excess capacity.

§ 5.5. Element layout

There are many ways to arrange the elements of a matrix in memory, the most common in C++ being row-major dense rectangular. In Fortran-based libraries, the two-dimensional arrays used to represent matrices are usually column-major. There are also special arrangements of elements for upper/lower triangular and banded diagonal matrices that are both row-major and column-major. These arrangements of elements have been well-known for many years, and libraries like LAPACK in the hands of a knowledgeable user can use them to implement code that is optimal in both time and space.

§ 5.6. Element access and indexing

In keeping with the goal of supporting a natural syntax, and in analogy with the indexing operations provided by the random-access standard library containers, it seems reasonable to provide both const and non-const indexing for reading and writing individual elements.

§ 5.7. Element type

C++ supports a relatively narrow range of arithmetic types, lacking direct support for arbitrary precision numbers and fixed-point numbers, among others. Libraries exist to implement these types, and they should not be precluded from use in a standard linear algebra library. It is possible that individual elements of a *MathObj* may allocate memory, and therefore an implementation cannot assume that element types have trivial constructors or destructors.

§ 5.8. Mixed-element-type expressions

In general, when multiple built-in arithmetic types are present in an arithmetical expression, the resulting type will have a precision greater than or equal to that of the type with greatest precision in the expression. In other words, to the greatest reasonable extent, information is preserved.

We contend that a similar principal should apply to expressions involving *MathObjs* where more than one element type is present. Arithmetic operations involving *MathObjs* should, to the greatest reasonable extent, preserve element-wise information.

For example, just as the result of multiplying a `float` by a `double` is a `double`, the result multiplying a matrix-of-`float` by a matrix-of-`double` should be a matrix-of-`double`. We call the process of determining the resulting element type *element promotion*.

§ 5.9. Mixed-engine expressions

In analogy with element type, *MathObj* expressions may include mixed storage management strategies, as implemented by their corresponding engine types. For example, consider the case of a fixed-size matrix multiplied by a dynamically-resizable matrix. What is the engine type of the resulting matrix?

Expression involving mixed engine types should not limit the availability of basic arithmetic operations. This means that there should be a mechanism for determining the engine type of the resulting from such expressions. We call the process of determining the resulting engine type *engine promotion*.

We contend that in most cases, the resulting engine type should be at least as "general" as the most "general" of the two engine types. For example, one could make the argument that a dynamically-resizable engine is more general than a fixed-size engine, and therefore the resulting engine type in an expression involving both these engine types should be a dynamically-resizable engine.

However, there are cases in which it may be possible to choose a more performant engine at compile time. For example, consider the case adding a fixed-size matrix and a dynamically-resizable matrix. Although size checking must be performed at run time, the resulting engine might be specified as fixed-size.

§ 5.10. Arithmetic customization

In pursuit of optimal performance, developers may want to customize specific arithmetic operations, such as matrix-matrix or matrix-vector multiplication. Customization might be based on things like element layout in memory, fixed-size -vs- dynamically resizable, special hardware capabilities, etc.

One such possible optimization is the use of multiple cores, perhaps distributed across a network, to carry out multiplication on very large pairs of matrices, particularly in situations where the operation is

used to produce a third matrix rather than modify one of the operands; the matrix multiplication operation is particularly amenable to this approach.

Developers may also wish to make use of SIMD intrinsics to enable parallel evaluation of matrix multiplication. This is common in game development environments where programs are written for very specific platforms, where the make and model of processor is well defined. This would impact on element layout and storage. Such work has already been demonstrated in paper N4454.

It is possible that two operands may be associated with different arithmetic customizations. We call the process of determining which of those two customizations to employ when performing the actual arithmetic operations *operation traits promotion*.

§ 5.11. Linear algebra and `constexpr`

The fundamental set of operations for linear algebra can all be implemented in terms of a subset of the algorithms defined in the `<algorithm>` header, all of which are marked `constexpr` since C++20. Matrix and vector initialization is of course also possible at compile time.

The arrival of `std::is_constant_evaluated` in C++20 makes it possible to offer a `constexpr` implementation of the operations, allowing customizations to defer to them in `constexpr` evaluations while taking the customization's notionally superior run-time path in alternate situations.

§ 6. Interface description

In this section, we describe the various types, operators, and functions comprising the proposed interface. The reader should note that the descriptions below are by no means ready for wording; rather, they are intended to foster further discussions and refinements, and to serve as a guide for hardy souls attempting to build implementations from this specification.

§ 6.1. Overview

At the highest level, the interface is divided into three broad categories:

1. **Engines**, which are implementation types that manage the resources associated with a *MothObj* instance, including memory ownership and lifetime, as well as element access; and,
2. **MathObjs**, which provide a unified interface intended to model a corresponding mathematical abstraction (vector, matrix);

3. **Operators**, which provide the desired mathematical syntax and carry out the promised arithmetic.

At a lower level, and somewhat behind the scenes, are a number of supporting traits types employed by the operators to determine the return type of the operator and perform the corresponding arithmetic operation. There are several such traits types:

- **Numeric traits** specify and test the properties of matrix element types. The proposed traits class template `std::math::number_traits` is one of two suggested customization points.
- **Element promotion traits** determine the resulting element type of an arithmetic operation involving two *elements*.
- **Engine promotion traits** determine the resulting engine type of an arithmetic operation involving *matrix* and/or *vector* objects. As part of that process, this traits type uses the element promotion traits to determine the element type of the resulting engine.
- **Arithmetic traits** determine the type and value of a *MathObj* resulting from an arithmetical operation. As part of that process, this traits type uses the engine promotion traits to determine the engine type of the resulting *MathObj*. Having determined the result type, the arithmetic traits also have a member function that carries out the actual computations.
- **Operation traits** act as a "container" for element promotion, engine promotion, and arithmetic traits. This traits type is a template parameter to the *MathObj* types, and provides a way to inform an operator of the set of available arithmetic traits to be used when deciding how to perform an arithmetic operation.
- **Operation selector traits** provide the means by which an arithmetic operator selects the operation traits that will perform the arithmetic. In the case where each operand has the same operation traits, the decision is simple. However, it is possible that the operands may be instantiated with different operation traits types, and so the operator uses the operation selector traits to decide which operation traits type to use for computing its result. The proposed traits class `std::math::matrix_operation_traits` is the second of two suggested customization points.

The following sections describe the building blocks in more detail, starting at the lowest level, and working upward in order of dependency.

A note on template parameter nomenclature: To avoid excessive visual noise in the code shown in the subsequent sections of this paper, we use the following abbreviation-based naming conventions for template parameters:

- Parameter names `T`, `T1`, and `T2` represent element types.
- Parameter names `ET`, `ET1`, and `ET2` represent engine types.

- Parameter names `OT`, `OT1`, and `OT2` represent operation traits types.
- Parameter names `OP`, `OP1`, and `OP2` represent the operand types deduced by an arithmetic operator.
- Parameter names `AT`, `AT1`, and `AT2` represent allocator types.
- Parameter names `N`, `N1`, and `N2` represent the number of elements in a fixed-size vector.
- Parameter names `C`, `C1`, and `C2` represent the number of columns in a fixed-size matrix.
- Parameter names `R`, `R1`, and `R2` represent the number of rows in a fixed-size matrix.

§ 6.2. `std::math` namespace

We propose adding the new namespace `std::math` to the standard library to contain the linear algebra facilities described in this paper. There are two reasons for this: first, in order to group a related set of mathematical facilities in a meaningful and orderly fashion; and second, to distinguish the mathematical vector type described in this proposal, `std::math::vector`, from the general-purpose container `std::vector`.

§ 6.3. Header `<linear_algebra>` synopsis

```
#include <cstdint>
#include <algorithm>
#include <complex>
#include <initializer_list>
#include <memory>
#include <numeric>
#include <tuple>
#include <type_traits>

namespace std::math {

    //- Some tags for specifying how engines should behave.
    //
    using scalar_engine_tag          = integral_constant<int, 0>;

    using const_vector_engine_tag    = integral_constant<int, 1>;
    using mutable_vector_engine_tag  = integral_constant<int, 2>;
    using resizable_vector_engine_tag = integral_constant<int, 3>;

    using const_matrix_engine_tag    = integral_constant<int, 4>;
    using mutable_matrix_engine_tag  = integral_constant<int, 5>;
    using resizable_matrix_engine_tag = integral_constant<int, 6>;

    //- A traits type that supplies important information about a numerical type.
    // This traits class is a customization point.
    //
    template<class T>    struct number_traits;

    //- Traits for verifying appropriate matrix element types.
    //
    template<class T>    struct is_complex;
    template<class T>    struct is_field;
    template<class T>    struct is_nc_ring;
    template<class T>    struct is_ring;
    template<class T>    struct is_matrix_element;

    template<class T>    inline constexpr bool is_complex_v = is_complex<T>::value;
    template<class T>    inline constexpr bool is_field_v   = is_field<T>::value;
    template<class T>    inline constexpr bool is_ring_v    = is_ring<T>::value;
    template<class T>    inline constexpr bool is_nc_ring_v = is_nc_ring<T>::value;
    template<class T>    inline constexpr bool is_matrix_element_v = is_matrix_element<T>::value;
```

```

//- Non-owning view-style engines.
//
template<class ET>                class matrix_column_view;
template<class ET>                class matrix_row_view;
template<class ET>                class matrix_transpose_view;

//- Owning engines with fixed-size internal storage.
//
template<class T, int32_t N>       class fs_vector_engine;
template<class T, int32_t R, int32_t C> class fs_matrix_engine;

//- Owning engines with dynamically-allocated external storage.
//
template<class T, class AT>       class dr_vector_engine;
template<class T, class AT>       class dr_matrix_engine;

//- The default element promotion, engine promotion, and arithmetic operations.
// the four basic arithmetic operations.
//
struct matrix_operation_traits;

//- Primary math object types.
//
template<class ET, class OT=matrix_operation_traits> class vector;
template<class ET, class OT=matrix_operation_traits> class matrix;

//- Math object element promotion traits, per arithmetical operation.
//
template<class T1>                struct matrix_negation_element_traits;
template<class T1, class T2>      struct matrix_addition_element_traits;
template<class T1, class T2>      struct matrix_subtraction_element_traits;
template<class T1, class T2>      struct matrix_multiplication_element_traits;

//- Math object engine promotion traits, per arithmetical operation.
//
template<class OT, class ET1>      struct matrix_negation_engine_traits;
template<class OT, class ET1, class ET2> struct matrix_addition_engine_traits;
template<class OT, class ET1, class ET2> struct matrix_subtraction_engine_traits;
template<class OT, class ET1, class ET2> struct matrix_multiplication_engine_traits;

//- Math object arithmetic traits.
//

```

```

template<class OT, class OP1>          struct matrix_negation_traits;
template<class OT, class OP1, class OP2> struct matrix_addition_traits;
template<class OT, class OP1, class OP2> struct matrix_subtraction_traits;
template<class OT, class OP1, class OP2> struct matrix_multiplication_traits;

//- A traits type that chooses between two operation traits types in the binary
// arithmetic operators and free functions that act like binary operators
// (e.g., outer_product()). This traits class is a customization point.
//
template<class OT1, class OT2>          struct matrix_operation_traits_selector;

//- Addition operators
//
template<class ET1, class OT1, class ET2, class OT2>
auto operator +(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2);

template<class ET1, class OT1, class ET2, class OT2>
auto operator +(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2);

//- Subtraction operators
//
template<class ET1, class OT1, class ET2, class OT2>
auto operator -(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2);

template<class ET1, class OT1, class ET2, class OT2>
auto operator -(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2);

//- Negation operators
//
template<class ET1, class OT1>
auto operator -(vector<ET1, OT1> const& v1);

template<class ET1, class OT1, class ET2, class OT2>
auto operator -(matrix<ET1, OT1> const& m1);

//- Vector*Scalar multiplication operators
//
template<class ET1, class OT1, class S2>
auto operator *(vector<ET1, OT1> const& v1, S2 const& s2);

template<class S1, class ET2, class OT2>
auto operator *(S1 const& s1, vector<ET2, OT2> const& v2);

```

```

//- Matrix*Scalar multiplication operators
//
template<class ET1, class OT1, class S2>
auto operator *(matrix<ET1, OT1> const& m1, S2 const& s2);

template<class S1, class ET2, class OT2>
auto operator *(S1 const& s1, matrix<ET2, OT2> const& m2);

//- Vector*Matrix multiplication operator
//
template<class ET1, class OT1, class ET2, class OT2>
auto operator *(vector<ET1, OT1> const& v1, matrix<ET2, OT2> const& m2);

//- Matrix*Vector multiplication operator
//
template<class ET1, class OT1, class ET2, class OT2>
auto operator *(matrix<ET1, OT1> const& m1, vector<ET2, OT2> const& v2);

//- Vector*Vector multiplication operator
//
template<class ET1, class OT1, class ET2, class OT2>
auto operator *(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2);

//- Matrix*Matrix multiplication operator
//
template<class ET1, class OT1, class ET2, class OT2>
auto operator *(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2);

//- Convenience aliases for vector and matrix objects based on dynamically-
//
template<class T, class AT = allocator<T>>
using dyn_vector = vector<dr_vector_engine<T, AT>, matrix_operation_traits>

template<class T, class AT = allocator<T>>
using dyn_matrix = matrix<dr_matrix_engine<T, AT>, matrix_operation_traits>

//- Convenience aliases for vector and matrix objects based on fixed-size e
//
template<class T, int32_t N>
using fs_vector = vector<fs_vector_engine<T, N>, matrix_operation_traits>;

template<class T, int32_t R, int32_t C>
using fs_matrix = matrix<fs_matrix_engine<T, R, C>, matrix_operation_traits>;

```

```
}
```

§ 6.4. Numeric traits

§ 6.4.1. `number_traits<T>`

Traits type `number_traits<T>` specifies certain properties of numeric types. It is a customization point intended to be specialized by the user when a new user-defined type is to be used with the library.

```
template<class T>
struct number_traits
{
    using is_field      = ...;    //- Implementation-defined
    using is_nc_ring    = ...;    //- Implementation-defined
    using is_ring       = ...;    //- Implementation-defined
};

template<class T>
struct number_traits<complex<T>> : public number_traits<T> {};
```

§ 6.4.2. `is_complex<T>`

Traits type `is_complex<T>` indicates whether its template parameter `T` is of type `std::complex<V>` for some (presumably) numeric type `V`. A possible implementation might be:

```
template<class T>
struct is_complex : public false_type
{};

template<class T>
struct is_complex<complex<T>> : public true_type
{};
```

§ 6.4.3. `is_field<T>`

Traits `is_field<T>` type indicates whether its template parameter `T` is a type that models a mathematical field. This proposal expects that `is_field` will be implemented in terms of `number_traits`, perhaps as follows:

```
template<class T>
struct is_field : public number_traits<T>::is_field
{};
```

§ 6.4.4. `is_ring<T>`

Traits type `is_ring<T>` indicates whether its template parameter `T` is a type that models a mathematical ring. This proposal expects that `is_ring` will be implemented in terms of `number_traits`, perhaps as follows:

```
template<class T>
struct is_ring : public number_traits<T>::is_ring
{};
```

§ 6.4.5. `is_nc_ring<T>`

Traits type `is_nc_ring<T>` indicates whether its template parameter `T` is a type that models a non-commutative ring. This proposal expects that `is_nc_ring` will be implemented in terms of `number_traits`, perhaps as follows:

```
template<class T>
struct is_nc_ring : public number_traits<T>::is_nc_ring
{};
```

§ 6.4.6. `is_matrix_element<T>`

Traits type `is_matrix_element<T>` indicates whether its template parameter `T` is a type that is suitable as matrix element type.

```
template<class T> struct is_matrix_element;
```

The purpose of this type is to constrain the set of element types used to instantiate a *MathObj* to those types which fulfill at least the requirements of non-commutative rings. For now, the constraint is imposed via static assertion; we expect this to change as appropriate concepts are developed and integrated into the library.

§ 6.5. Engine Types

The over-arching purpose of the engine types is to perform resource management on behalf of an associated, owning *MathObj* instance. At a minimum, all of the engine types provide a basic interface for const element indexing, determining row and column sizes, and determining row and column capacities. They also export public type aliases which specify their element type, whether or not they are dense, whether or not they are rectangular, whether or not they are resizable, whether or not their memory layout is row-major, and a 2-tuple for describing sizes and capacities.

It is important to note that an engine's resource management duties are primarily related to storage. To that end, an engine may own the storage it manages and control its lifetime, or it may be non-owning and represent a view of storage owned by some other object.

One can also imagine engines that manage resources related to execution. This is an area of ongoing work and not yet addressed in this proposal.

§ 6.5.1. `matrix_column_view<ET>`

Class template `matrix_column_view<ET>` implements an engine that is a non-owning, const view type implementing the basic vector engine interface. Its purpose is to provide a view of single column of a `matrix` object.

```

template<class ET>
class matrix_column_view
{
public:
    using engine_type      = ET;
    using engine_category  = const_vector_engine_tag;
    using element_type     = typename engine_type::element_type;
    using value_type       = typename engine_type::value_type;
    using reference        = typename engine_type::const_reference;
    using pointer          = typename engine_type::const_pointer;
    using const_reference  = typename engine_type::const_reference;
    using const_pointer    = typename engine_type::const_pointer;
    using iterator         = ...;                //- Implementation-defined
    using const_iterator   = ...;                //- Implementation-defined
    using difference_type  = typename engine_type::difference_type;
    using index_type       = typename engine_type::index_type;
    using size_type        = typename engine_type::size_type;

    using is_fixed_size    = typename engine_type::is_fixed_size;
    using is_resizable     = false_type;

    using is_column_major  = typename engine_type::is_row_major;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_row_major     = typename engine_type::is_column_major;

public:
    constexpr matrix_column_view();
    constexpr matrix_column_view(engine_type const& eng, index_type col);
    constexpr matrix_column_view(matrix_column_view&&) noexcept = default;
    constexpr matrix_column_view(matrix_column_view const&) = default;

    constexpr matrix_column_view& operator=(matrix_column_view&&) noexcept = default;
    constexpr matrix_column_view& operator=(matrix_column_view const&) = default;

    constexpr const_reference operator()(index_type i) const;
    constexpr const_iterator begin() const noexcept;
    constexpr const_iterator end() const noexcept;
    constexpr size_type capacity() const noexcept;
    constexpr size_type elements() const noexcept;

```

```
constexpr void      assign(matrix_column_view const& rhs);  
};
```

§ 6.5.2. `matrix_row_view<ET>`

Class template `matrix_row_view<ET>` implements an engine that is a non-owning, const view type implementing the basic vector engine interface. Its purpose is to provide a view of single row of a `matrix` object.

```

template<class ET>
class matrix_row_view
{
public:
    using engine_type      = ET;
    using engine_category  = const_vector_engine_tag;
    using element_type     = typename engine_type::element_type;
    using value_type       = typename engine_type::value_type;
    using reference        = typename engine_type::const_reference;
    using pointer          = typename engine_type::const_pointer;
    using const_reference  = typename engine_type::const_reference;
    using const_pointer    = typename engine_type::const_pointer;
    using iterator         = ...;                //- Implementation-defined
    using const_iterator   = ...;                //- Implementation-defined
    using difference_type  = typename engine_type::difference_type;
    using index_type       = typename engine_type::index_type;
    using size_type        = typename engine_type::size_type;

    using is_fixed_size    = typename engine_type::is_fixed_size;
    using is_resizable     = false_type;

    using is_column_major  = typename engine_type::is_row_major;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_row_major     = typename engine_type::is_column_major;

public:
    constexpr matrix_row_view();
    constexpr matrix_row_view(engine_type const& eng, index_type row);
    constexpr matrix_row_view(matrix_row_view&&) noexcept = default;
    constexpr matrix_row_view(matrix_row_view const&) = default;

    constexpr matrix_row_view& operator =(matrix_row_view&&) noexcept = de
    constexpr matrix_row_view& operator =(matrix_row_view const&) = default

    constexpr const_reference operator ()(index_type j) const;
    constexpr const_iterator begin() const noexcept;
    constexpr const_iterator end() const noexcept;
    constexpr size_type capacity() const noexcept;
    constexpr size_type elements() const noexcept;

```

```
constexpr void      assign(matrix_row_view const& rhs);  
};
```

§ 6.5.3. `matrix_transpose_view<ET>`

Class template `matrix_transpose_view<ET>` implements an engine that is a non-owning, const view type implementing the basic matrix engine interface. Its purpose is to provide a view of a `matrix` object that is the transpose of the `matrix`.

```

template<class ET>
class matrix_transpose_view
{
public:
    using engine_type      = ET;
    using engine_category  = const_matrix_engine_tag;
    using element_type     = typename engine_type::element_type;
    using value_type       = typename engine_type::value_type;
    using reference        = typename engine_type::const_reference;
    using pointer          = typename engine_type::const_pointer;
    using const_reference  = typename engine_type::const_reference;
    using const_pointer    = typename engine_type::const_pointer;
    using difference_type  = typename engine_type::difference_type;
    using index_type       = typename engine_type::index_type;
    using size_type        = typename engine_type::size_type;
    using size_tuple       = typename engine_type::size_tuple;

    using is_fixed_size    = typename engine_type::is_fixed_size;
    using is_resizable     = false_type;

    using is_column_major  = typename engine_type::is_row_major;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_row_major     = typename engine_type::is_column_major;

    using column_view_type = matrix_column_view<matrix_transpose_view>;
    using row_view_type    = matrix_row_view<matrix_transpose_view>;
    using transpose_view_type = matrix_transpose_view<matrix_transpose_view>;

public:
    constexpr matrix_transpose_view();
    constexpr matrix_transpose_view(engine_type const& eng);
    constexpr matrix_transpose_view(matrix_transpose_view&&) noexcept = default;
    constexpr matrix_transpose_view(matrix_transpose_view const&) = default;

    constexpr matrix_transpose_view& operator=(matrix_transpose_view&&)
    constexpr matrix_transpose_view& operator=(matrix_transpose_view const&)

    constexpr const_reference operator()(index_type i, index_type j) const

    constexpr size_type columns() const noexcept;
    constexpr size_type rows() const noexcept;
    constexpr size_tuple size() const noexcept;

```

```
constexpr size_type    column_capacity() const noexcept;  
constexpr size_type    row_capacity() const noexcept;  
constexpr size_tuple    capacity() const noexcept;  
  
constexpr void assign(matrix_transpose_view const& rhs);  
};
```

§ 6.5.4. `fs_vector_engine<T, N>`

Class template `fs_vector_engine<T, N>` implements a fixed-size, fixed-capacity engine for vectors having `N` elements. In addition to the basic vector engine interface, it provides member functions for mutable element indexing, swapping engine contents, and swapping individual elements.

```

template<class T, int32_t N>
class fs_vector_engine
{
    static_assert(N >= 1);

public:
    using engine_category = mutable_vector_engine_tag;
    using element_type     = T;
    using value_type       = T;
    using reference         = T&;
    using pointer           = T*;
    using const_reference   = T const&;
    using const_pointer     = T const*;
    using iterator          = ...;           //- Implementation-defined
    using const_iterator    = ...;          //- Implementation-defined
    using difference_type   = ptrdiff_t;
    using index_type        = int_fast32_t;
    using size_type         = int_fast32_t;

    using is_fixed_size     = true_type;
    using is_resizable      = false_type;

    using is_column_major   = true_type;
    using is_dense          = true_type;
    using is_rectangular    = true_type;
    using is_row_major      = true_type;

public:
    constexpr fs_vector_engine();
    template<class U>
    constexpr fs_vector_engine(initializer_list<U> list);
    constexpr fs_vector_engine(fs_vector_engine&&) noexcept = default;
    constexpr fs_vector_engine(fs_vector_engine const&) = default;

    constexpr fs_vector_engine& operator =(fs_vector_engine&&) noexcept
    constexpr fs_vector_engine& operator =(fs_vector_engine const&) = c

    constexpr const_reference operator ()(index_type i) const;
    constexpr const_iterator begin() const noexcept;
    constexpr const_iterator end() const noexcept;

    constexpr size_type capacity() const noexcept;
    constexpr index_type elements() const noexcept;

```

```

constexpr size_type      size() const noexcept;

constexpr reference      operator ()(index_type i);
constexpr iterator       begin() noexcept;
constexpr iterator       end() noexcept;

constexpr void swap(fs_vector_engine& rhs) noexcept;
constexpr void swap_elements(index_type i, index_type j) noexcept;
};

```

§ 6.5.5. `fs_matrix_engine<T, R, C>`

Class template `fs_matrix_engine<T, R, C>` implements a fixed-size, fixed-capacity engine for matrices having `R` rows and `C` columns. In addition to the basic engine interface, it provides member functions for mutable element indexing, swapping engine contents, swapping columns, and swapping rows.

```

template<class T, size_t R, size_t C>
class fs_matrix_engine
{
public:
    using engine_category = mutable_matrix_engine_tag;
    using element_type     = T;
    using value_type       = T;
    using reference        = T&;
    using pointer          = T*;
    using const_reference  = T const&;
    using const_pointer    = T const*;
    using difference_type  = ptrdiff_t;
    using index_type       = int_fast32_t;
    using size_type        = int_fast32_t;
    using size_tuple       = tuple<size_type, size_type>;

    using is_fixed_size    = true_type;
    using is_resizable     = false_type;

    using is_column_major  = false_type;
    using is_dense         = true_type;
    using is_rectangular   = true_type;
    using is_row_major     = true_type;

    using column_view_type = matrix_column_view<fs_matrix_engine>;
    using row_view_type    = matrix_row_view<fs_matrix_engine>;
    using transpose_view_type = matrix_transpose_view<fs_matrix_engine>;

public:
    constexpr fs_matrix_engine();
    template<class U>
    constexpr fs_matrix_engine(initializer_list<U> list);
    constexpr fs_matrix_engine(fs_matrix_engine&&) noexcept = default;
    constexpr fs_matrix_engine(fs_matrix_engine const&) = default;

    constexpr fs_matrix_engine& operator =(fs_matrix_engine&&) noexcept
    constexpr fs_matrix_engine& operator =(fs_matrix_engine const&) = c

    constexpr const_reference operator ()(index_type i, index_type j) cor

    constexpr index_type columns() const noexcept;
    constexpr index_type rows() const noexcept;
    constexpr size_tuple size() const noexcept;

```

```

constexpr size_type    column_capacity() const noexcept;
constexpr size_type    row_capacity() const noexcept;
constexpr size_tuple    capacity() const noexcept;

constexpr reference     operator ()(index_type i, index_type j);

constexpr void          assign(fs_matrix_engine const& rhs);
template<class ET2>
constexpr void          assign(ET2 const& rhs);

constexpr void          swap(fs_matrix_engine& rhs) noexcept;
constexpr void          swap_columns(index_type j1, index_type j2) noexcept;
constexpr void          swap_rows(index_type i1, index_type i2) noexcept;
};

```

§ 6.5.6. `dr_vector_engine<T, AT>`

Class template `dr_vector_engine<T, AT>` implements an engine for vectors whose sizes and capacities can be changed at runtime. In addition to the basic engine interface, it provides member functions for mutable element indexing, swapping engine contents, swapping individual elements, and resizing.

```

template<class T, class AT=std::allocator<T>>
class dr_vector_engine
{
    static_assert(is_matrix_element_v<T>);

public:
    using engine_category = resizable_vector_engine_tag;
    using element_type     = T;
    using value_type       = T;
    using alloc_type       = AT;
    using reference        = T&;
    using pointer          = typename allocator_traits<AT>::pointer;
    using const_reference  = T const&;
    using const_pointer    = typename allocator_traits<AT>::const_pointer;
    using iterator         = ...;                //- Implementation-defined
    using const_iterator   = ...;                //- Implementation-defined
    using difference_type  = ptrdiff_t;
    using index_type       = ptrdiff_t;
    using size_type        = ptrdiff_t;

    using is_fixed_size    = false_type;
    using is_resizable     = true_type;

    using is_column_major  = true_type;
    using is_dense         = true_type;
    using is_rectangular   = true_type;
    using is_row_major     = true_type;

public:
    ~dr_vector_engine() noexcept;
    dr_vector_engine();
    template<class U>
    constexpr dr_vector_engine(initializer_list<U> list);
    dr_vector_engine(dr_vector_engine&&) noexcept;
    dr_vector_engine(dr_vector_engine const&);
    dr_vector_engine(size_type elems);
    dr_vector_engine(size_type elems, size_type elem_cap);

    dr_vector_engine& operator =(dr_vector_engine&&) noexcept;
    dr_vector_engine& operator =(dr_vector_engine const&);

    const_reference operator ()(index_type i) const;
    const_iterator begin() const noexcept;

```

```

    const_iterator end() const noexcept;

    size_type      elements() const noexcept;
    size_type      capacity() const noexcept;

    reference      operator ()(index_type i);
    iterator       begin() noexcept;
    iterator       end() noexcept;

    void           assign(dr_vector_engine const& rhs);
    template<class ET2>
    void           assign(ET2 const& rhs);

    void           reserve(size_type cap);
    void           resize(size_type elems);
    void           resize(size_type elems, size_type cap);
    void           swap(dr_vector_engine& rhs) noexcept;
    void           swap_elements(index_type i, index_type j) noexcept;
};

```

§ 6.5.7. `dr_matrix_engine<T, AT>`

Class template `dr_matrix_engine<T, AT>` implements an engine for matrices whose sizes and capacities can be changed at runtime. In addition to the basic engine interface, it provides member functions for mutable element indexing, swapping engine contents, swapping columns, swapping rows, and resizing.

```

template<class T, class AT=std::allocator<T>>
class dr_matrix_engine
{
    static_assert(is_matrix_element_v<T>);

public:
    using engine_category = resizable_matrix_engine_tag;
    using element_type     = T;
    using value_type       = T;
    using allocator_type    = AT;
    using reference         = T&;
    using pointer           = typename allocator_traits<AT>::pointer;
    using const_reference   = T const&;
    using const_pointer     = typename allocator_traits<AT>::const_pointer;
    using difference_type   = ptrdiff_t;
    using index_type        = ptrdiff_t;
    using size_type         = ptrdiff_t;
    using size_tuple        = tuple<size_type, size_type>;

    using is_fixed_size     = false_type;
    using is_resizable      = true_type;

    using is_column_major   = false_type;
    using is_dense          = true_type;
    using is_rectangular    = true_type;
    using is_row_major      = true_type;

    using column_view_type  = matrix_column_view<dr_matrix_engine>;
    using row_view_type     = matrix_row_view<dr_matrix_engine>;
    using transpose_view_type = matrix_transpose_view<dr_matrix_engine>;

public:
    ~dr_matrix_engine();
    dr_matrix_engine();
    dr_matrix_engine(dr_matrix_engine&& rhs) noexcept;
    dr_matrix_engine(dr_matrix_engine const& rhs);
    dr_matrix_engine(size_type rows, size_type cols);
    dr_matrix_engine(size_type rows, size_type cols, size_type rowcap, size_type colcap);

    dr_matrix_engine& operator =(dr_matrix_engine&&) noexcept;
    dr_matrix_engine& operator =(dr_matrix_engine const&);

    const_reference      operator ()(index_type i, index_type j) const;

```

```

size_type    columns() const noexcept;
size_type    rows() const noexcept;
size_tuple   size() const noexcept;

size_type    column_capacity() const noexcept;
size_type    row_capacity() const noexcept;
size_tuple   capacity() const noexcept;

reference    operator()(index_type i, index_type j);

void         assign(dr_matrix_engine const& rhs);
template<class ET2>
void         assign(ET2 const& rhs);

void         reserve(size_type rowcap, size_type colcap);
void         resize(size_type rows, size_type cols);
void         resize(size_type rows, size_type cols, size_type rowcap, size_t
void         swap(dr_matrix_engine& other) noexcept;
void         swap_columns(index_type c1, index_type c2) noexcept;
void         swap_rows(index_type r1, index_type r2) noexcept;
};

```

§ 6.6. Math object types

§ 6.6.1. `vector<ET, OT>`

Class template `vector<ET, OT>` represents a vector, with element type and resource management implemented by the engine type `ET`, and arithmetic operations specified by the operation traits type `OT`. If the underlying engine type provides dynamic resizing, then this class will as well.

```

template<class ET, class OT>
class vector
{
    static_assert(detail::is_vector_engine_v<ET>);
    static_assert(is_matrix_element_v<typename ET::element_type>);

public:
    using engine_type      = ET;
    using element_type     = typename engine_type::element_type;
    using reference        = typename engine_type::reference;
    using pointer          = typename engine_type::pointer;
    using const_reference  = typename engine_type::const_reference;
    using const_pointer    = typename engine_type::const_pointer;
    using iterator         = typename engine_type::iterator;
    using const_iterator   = typename engine_type::const_iterator;
    using index_type       = typename engine_type::index_type;
    using size_type        = typename engine_type::size_type;

    using transpose_type   = vector const&;
    using hermitian_type   = conditional_t<is_complex_v<element_type>, vector, vector const&>;

    using is_fixed_size    = typename engine_type::is_fixed_size;
    using is_resizable     = typename engine_type::is_resizable;

    using is_column_major  = typename engine_type::is_column_major;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_row_major     = typename engine_type::is_row_major;

public:
    ~vector() = default;

    constexpr vector();
    constexpr vector(vector&&) noexcept = default;
    constexpr vector(vector const&) = default;
    template<class ET2, class OT2>
    constexpr vector(vector<ET2, OT2> const& src);

    template<class U>
    constexpr vector(initializer_list<U> list);
    template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
    constexpr vector(size_type elems);
    template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>

```

```

constexpr vector(size_type elems, size_type elemcap);

constexpr vector& operator =(vector&&) noexcept = default;
constexpr vector& operator =(vector const&) = default;
template<class ET2, class OT2>
constexpr vector& operator =(vector<ET2, OT2> const& rhs);

//- Const element access.
//
constexpr const_reference    operator ()(index_type i) const;
constexpr const_iterator    begin() const noexcept;
constexpr const_iterator    end() const noexcept;

//- Accessors.
//
constexpr size_type        capacity() const noexcept;
constexpr index_type      elements() const noexcept;
constexpr size_type        size() const noexcept;

//- Transpose and Hermitian.
//
constexpr transpose_type    t() const;
constexpr hermitian_type    h() const;

//- Mutable element access.
//
constexpr reference        operator ()(index_type i);
constexpr iterator        begin() noexcept;
constexpr iterator        end() noexcept;

//- Assignment.
//
constexpr void            assign(vector const& rhs);
template<class ET2, class OT2>
constexpr void            assign(vector<ET2, OT2> const& rhs);

//- Change capacity.
//
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr void            reserve(size_type elemcap);

//- Change size.
//

```

```

template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr void      resize(size_type elems);

//- Change size and capacity in one shot.
//
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr void      resize(size_type elems, size_type elemcap);

//- Element operations.
//
constexpr void      swap(vector& rhs) noexcept;
constexpr void      swap_elements(index_type i, index_type j) noexcept;
};

```

§ 6.6.2. `matrix<ET, OT>`

Class template `matrix<ET, OT>` represents a matrix, with element type and resource management implemented by the engine type `ET`, and arithmetic operations specified by the operation traits type `OT`. If the underlying engine type provides dynamic resizing, then this class will as well.

```

template<class ET, class OT>
class matrix
{
    static_assert(detail::is_matrix_engine_v<ET>);

public:
    using engine_type      = ET;
    using element_type     = typename engine_type::element_type;
    using reference        = typename engine_type::reference;
    using const_reference  = typename engine_type::const_reference;
    using index_type       = typename engine_type::index_type;
    using size_type        = typename engine_type::size_type;
    using size_tuple       = typename engine_type::size_tuple;

    using column_type      = vector<matrix_column_view<engine_type>, OT>;
    using row_type         = vector<matrix_row_view<engine_type>, OT>;
    using transpose_type   = matrix<matrix_transpose_view<engine_type>, OT>;
    using hermitian_type   = conditional_t<is_complex_v<element_type>, matrix<matrix_transpose_view<engine_type>, OT>, matrix>;

    using is_fixed_size    = typename engine_type::is_fixed_size;
    using is_resizable     = typename engine_type::is_resizable;

    using is_column_major  = typename engine_type::is_column_major;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_row_major     = typename engine_type::is_row_major;

public:
    ~matrix() = default;
    constexpr matrix() = default;
    constexpr matrix(matrix&&) noexcept = default;
    constexpr matrix(matrix const&) = default;

    template<class U, class ET2 = ET, detail::enable_if_fixed_size<ET, ET2>
    constexpr matrix(initializer_list<U> list);
    template<class ET2, class OT2>
    constexpr matrix(matrix<ET2, OT2> const& src);
    template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
    constexpr matrix(size_tuple size);
    template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
    constexpr matrix(size_type rows, size_type cols);

    template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>

```

```

constexpr matrix(size_tuple size, size_tuple cap);
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr matrix(size_type rows, size_type cols, size_type rowcap, size

constexpr matrix&      operator =(matrix&&) noexcept = default;
constexpr matrix&      operator =(matrix const&) = default;
template<class ET2, class OT2>
constexpr matrix&      operator =(matrix<ET2, OT2> const& rhs);

//- Const element access.
//
constexpr const_reference  operator ()(index_type i, index_type j) con

//- Accessors.
//
constexpr index_type      columns() const noexcept;
constexpr index_type      rows() const noexcept;
constexpr size_tuple      size() const noexcept;

constexpr size_type       column_capacity() const noexcept;
constexpr size_type       row_capacity() const noexcept;
constexpr size_tuple      capacity() const noexcept;

//- Column view, row view, transpose view, and Hermitian.
//
constexpr column_type      column(index_type j) const noexcept;
constexpr row_type         row(index_type i) const noexcept;
constexpr transpose_type   t() const;
constexpr hermitian_type   h() const;

//- Mutable element access.
//
constexpr reference        operator ()(index_type i, index_type j);

//- Assignment.
//
constexpr void             assign(matrix const& rhs);
template<class ET2, class OT2>
constexpr void             assign(matrix<ET2, OT2> const& rhs);

//- Change capacity.
//
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>

```

```

constexpr void      reserve(size_tuple cap);
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr void      reserve(size_type rowcap, size_type colcap);

//- Change size.
//
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr void      resize(size_tuple size);
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr void      resize(size_type rows, size_type cols);

//- Change size and capacity in one shot.
//
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr void      resize(size_tuple size, size_tuple cap);
template<class ET2 = ET, detail::enable_if_resizable<ET, ET2> = true>
constexpr void      resize(size_type rows, size_type cols, size_type rcap, size_type ccap);

//- Swapping operations.
//
template<class ET2 = ET, detail::enable_if_mutable<ET, ET2> = true>
constexpr void      swap(matrix& rhs) noexcept;
template<class ET2 = ET, detail::enable_if_mutable<ET, ET2> = true>
constexpr void      swap_columns(index_type i, index_type j) noexcept;
template<class ET2 = ET, detail::enable_if_mutable<ET, ET2> = true>
constexpr void      swap_rows(index_type i, index_type j) noexcept;
};

```

§ 6.7. Operation traits

§ 6.7.1. `matrix_operation_traits`

Class `matrix_operation_traits` is a traits-style template parameter to `vector` and `matrix`. Its purpose is to associate sets of element promotion traits, engine promotion traits, and arithmetic traits with a *MathObj* so that those traits may be conveyed into an arithmetic operator.

```

struct matrix_operation_traits
{
    //- Default element promotion traits.
    //
    template<class T1>
    using element_negation_traits = matrix_negation_element_traits<T1>;

    template<class T1, class T2>
    using element_addition_traits = matrix_addition_element_traits<T1, T2>;

    template<class T1, class T2>
    using element_subtraction_traits = matrix_subtraction_element_traits<T1, T2>;

    template<class T1, class T2>
    using element_multiplication_traits = matrix_multiplication_element_traits<T1, T2>;

    //- Default engine promotion traits.
    //
    template<class OTR, class ET1>
    using engine_negation_traits = matrix_negation_engine_traits<OTR, ET1>;

    template<class OTR, class ET1, class ET2>
    using engine_addition_traits = matrix_addition_engine_traits<OTR, ET1, ET2>;

    template<class OTR, class ET1, class ET2>
    using engine_subtraction_traits = matrix_subtraction_engine_traits<OTR, ET1, ET2>;

    template<class OTR, class ET1, class ET2>
    using engine_multiplication_traits = matrix_multiplication_engine_traits<OTR, ET1, ET2>;

    //- Default arithmetic operation traits.
    //
    template<class OP1, class OTR>
    using negation_traits = matrix_negation_traits<OP1, OTR>;

    template<class OTR, class OP1, class OP2>
    using addition_traits = matrix_addition_traits<OTR, OP1, OP2>;

    template<class OTR, class OP1, class OP2>
    using subtraction_traits = matrix_subtraction_traits<OTR, OP1, OP2>;

    template<class OTR, class OP1, class OP2>
    using multiplication_traits = matrix_multiplication_traits<OTR, OP1, OP2>;
}

```

```
using multiplication_traits = matrix_multiplication_traits<OTR, OP1, OF  
};
```

This traits type is a customization point. Users may override the default functionality it provides by creating a custom operation traits class in their own namespace, and defining only those members necessary to implement the desired custom behavior.

§ 6.7.2. `matrix_operation_traits_selector<OT1, OT2>`

Class template `matrix_operation_traits_selector<OT1, OT2>` is used by the binary arithmetic operators to select the operation traits type to used in performing an arithmetic operation. The selection is based on the operation traits types of the two operands.

```

//- Primary template and expected specializations.
//
template<class T1, class T2>
struct matrix_operation_traits_selector;

template<class T1>
struct matrix_operation_traits_selector<T1, T1>
{
    using traits_type = T1;
};

template<class T1>
struct matrix_operation_traits_selector<T1, matrix_operation_traits>
{
    using traits_type = T1;
};

template<class T1>
struct matrix_operation_traits_selector<matrix_operation_traits, T1>
{
    using traits_type = T1;
};

template<>
struct matrix_operation_traits_selector<matrix_operation_traits, matrix_operation_traits>
{
    using traits_type = matrix_operation_traits;
};

//- Convenience alias.
//
template<class T1, class T2>
using matrix_operation_traits_selector_t = typename matrix_operation_traits_selector<T1, T2>::traits_type;

```

§ 6.8. Element promotion traits

Element promotion traits are used by the library to determine the resulting element type of an arithmetical expression having one or two *MathObj* operands.

§ 6.8.1. `matrix_negation_element_traits<T1>`

Class template `matrix_negation_element_traits<T1>` implements the default traits type for determining the element type of the *MathObj* instance resulting from negating a given *MathObj* instance.

Alias template `matrix_negation_element_t<OT, T1, T2>` is used by the library to return the nested type `OT::element_negation_traits<T1>`.

```
template<class T1>
struct matrix_negation_element_traits
{
    using element_type = decltype(-declval<T1>());
};

template<class OT, class T1>
using matrix_negation_element_t = ...;           //- Implementation-defined
```

§ 6.8.2. `matrix_addition_element_traits<T1, T2>`

Class template `matrix_addition_element_traits<T1, T2>` implements the default traits type for determining the element type of a *MathObj* instance resulting from the addition of two other *MathOb* instances.

Alias template `matrix_addition_element_t<OT, T1, T2>` is used by the library to obtain the nested type `OT::element_addition_traits<T1, T2>`.

```
template<class T1, class T2>
struct matrix_addition_element_traits
{
    using element_type = decltype(declval<T1>() + declval<T2>());
};

template<class OT, class T1, class T2>
using matrix_addition_element_t = ...;           //- Implementation-defined
```

§ 6.8.3. `matrix_subtraction_element_traits<T1, T2>`

Class template `matrix_subtraction_element_traits<T1, T2>` implements the default traits type for determining the element type of a *MathObj* instance resulting from the subtraction of two other *MathOb* instances.

Alias template `matrix_subtraction_element_t<OT, T1, T2>` is used by the library to obtain the nested type `OT::element_subtraction_traits<T1, T2>`.

```
template<class T1, class T2>
struct matrix_subtraction_element_traits
{
    using element_type = decltype(declval<T1>() - declval<T2>());
};

template<class OT, class T1, class T2>
using matrix_subtraction_element_t = ...;           //- Implementation-defined
```

§ 6.8.4. `matrix_multiplication_element_traits<T1, T2>`

Class template `matrix_multiplication_element_traits<T1, T2>` implements the default traits type for determining the element type of a *MathObj* instance resulting from the multiplication of two other *MathOb* instances.

Alias template `matrix_multiplication_element_t<OT, T1, T2>` is used by the library to obtain the nested type `OT::element_multiplication_traits<T1, T2>`.

```
template<class T1, class T2>
struct matrix_multiplication_element_traits
{
    using element_type = decltype(declval<T1>() * declval<T2>());
};

template<class OT, class T1, class T2>
using matrix_multiplication_element_t = ...;       //- Implementation-defined
```

§ 6.9. Engine promotion traits

Engine promotion traits are used by the arithmetic traits to determine the resulting engine types in an arithmetical expression.

§ 6.9.1. `matrix_negation_engine_traits<OT, ET1>`

Class template `matrix_negation_engine_traits<OT, ET1>` implements a traits type that determines the resulting engine type when negating a *MathObj*.

Alias template `matrix_negation_engine_t<OT, ET1>` is used by the library to obtain the nested type `OT::engine_negation_traits<ET1>`.

```
template<class OT, class ET1>
struct matrix_negation_engine_traits
{
    using element_type = matrix_negation_element_t<OT, typename ET1::element_type>;

    using engine_type = ...;                //- Implementation-defined
};

template<class OT, class ET1>
using matrix_negation_engine_t = ...;      //- Implementation-defined
```

§ 6.9.2. `matrix_addition_engine_traits<OT, ET1, ET2>`

Class template `matrix_addition_engine_traits<OT, ET1, ET2>` implements a traits type that determines the resulting engine type when adding two compatible *MathObjs*.

Alias template `matrix_addition_engine_t<OT, ET1, ET2>` is used by the library to obtain the nested type `OT::element_addition_traits<ET1, ET2>`.

```

template<class OT, class ET1, class ET2>
struct matrix_addition_engine_traits
{
    using element_type = matrix_addition_element_t<OT,
                                                    typename ET1::element_ty
                                                    typename ET2::element_ty

    using engine_type = ...;                      //- Implementation-defined
};

template<class OT, class ET1, class ET2>
using matrix_addition_engine_t = detail::engine_add_type_t<OT, ET1, ET2>;

```

§ 6.9.3. `matrix_subtraction_engine_traits<OT, ET1, ET2>`

Class template `matrix_subtraction_engine_traits<OT, ET1, ET2>` implements a traits type that determines the resulting engine type when subtracting two compatible *MathObjs*.

Alias template `matrix_subtraction_engine_t<OT, ET1, ET2>` is used by the library to obtain the nested type `OT::element_subtraction_traits<ET1, ET2>`.

```

template<class OT, class ET1, class ET2>
struct matrix_subtraction_engine_traits
{
    using element_type = matrix_subtraction_element_t<OT,
                                                    typename ET1::element
                                                    typename ET2::element

    using engine_type = ...;                      //- Implementation-defined
};

template<class OT, class ET1, class ET2>
using matrix_subtraction_engine_t = ...;          //- Implementation-defined

```

§ 6.9.4. `matrix_multiplication_engine_traits<OT, ET1, ET2>`

Class template `matrix_multiplication_engine_traits<OT, ET1, ET2>` implements a traits type that determines the resulting engine type when multiplying two compatible *MathObjs*.

Alias template `matrix_multiplication_engine_t<OT, ET1, ET2>` is used by the library to obtain the nested type `OT::element_multiplication_traits<ET1, ET2>`.

```
template<class OT, class ET1, class ET2>
struct matrix_multiplication_engine_traits
{
    using element_type = matrix_multiplication_element_t<OT,
                                                         typename ET1::element_type,
                                                         typename ET2::element_type>;

    using engine_type = ...;                               //- Implementation-defined.
};

template<class OT, class ET1, class ET2>
using matrix_multiplication_engine_t = ...;                //- Implementation-defined
```

§ 6.10. Arithmetic traits

This section defines a set of arithmetic traits types for negation, addition, subtraction, and multiplication. The purpose of each of these traits types is threefold:

1. to determine the element type of the resulting *MathObj*;
2. to determine the engine type of the resulting *MathObj*; and
3. to carry out the arithmetical operation and return its result.

The idea here is that arithmetic operators (described below) simply forward to the appropriate traits type, which does the heavy lifting.

§ 6.10.1. `matrix_negation_traits<OT, OP1>`

Class template `matrix_negation_traits<OT, OP1>` is an arithmetic traits type that performs the negation of a *MathObj* and returns the result in another *MathObj* having an implementation-defined

engine type. There are two partial specializations to support the two overloaded negation operators described below.

Alias template `matrix_negation_traits_t<OT, OP1>` is used by the library to obtain the nested type `OT::negation_traits<OP1>`.

```
template<class OT, class ET1, class OT1>
struct matrix_negation_traits<OT, vector<ET1, OT1>>
{
    using engine_type = matrix_negation_engine_t<OT, ET1>;
    using op_traits    = OT;
    using result_type  = vector<engine_type, op_traits>;

    static result_type negate(vector<ET1, OT1> const& v1);
};

template<class OT, class ET1, class OT1>
struct matrix_negation_traits<OT, matrix<ET1, OT1>>
{
    using engine_type = matrix_negation_engine_t<OT, ET1>;
    using op_traits    = OT;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type negate(matrix<ET1, OT1> const& m1);
};

template<class OT, class OP1>
using matrix_negation_traits_t = ...;           //- Implementation-defined
```

§ 6.10.2. `matrix_addition_traits<OT, OP1, OP2>`

Class template `matrix_addition_traits<OT, OP1, OP2>` is an arithmetic traits type that performs the addition of two compatible *MathObjs* and returns the result in a *MathObj* having an implementation-defined engine type. There are two partial specializations to support the two overloaded addition operators described below.

Alias template `matrix_addition_traits_t<OT, OP1, OP2>` is used by the library to obtain the nested type `OT::addition_traits<OP1, OP2>`.

```

template<class OT, class ET1, class OT1, class ET2, class OT2>
struct matrix_addition_traits<OT, vector<ET1, OT1>, vector<ET2, OT2>>
{
    using engine_type = matrix_addition_engine_t<OT, ET1, ET2>;
    using op_traits    = OT;
    using result_type  = vector<engine_type, op_traits>;

    static result_type add(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2)
    {
        return v1 + v2;
    }
};

template<class OT, class ET1, class OT1, class ET2, class OT2>
struct matrix_addition_traits<OT, matrix<ET1, OT1>, matrix<ET2, OT2>>
{
    using engine_type = matrix_addition_engine_t<OT, ET1, ET2>;
    using op_traits    = OT;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type add(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2)
    {
        return m1 + m2;
    }
};

template<class OT, class OP1, class OP2>
using matrix_addition_traits_t = detail::addition_traits_t<OT, OP1, OP2>;

```

§ 6.10.3. `matrix_subtraction_traits<OT, OP1, OP2>`

Class template `matrix_subtraction_traits<OT, OP1, OP2>` is an arithmetic traits type that performs the subtraction of two compatible *MathObjs* and returns the result in a *MathObj* having an implementation-defined engine type. There are two partial specializations to support the two overloaded subtraction operators described below.

Alias template `matrix_subtraction_traits_t<OT, OP1, OP2>` is used by the library to obtain the nested type `OT::subtraction_traits<OP1, OP2>`.

```

template<class OT, class ET1, class OT1, class ET2, class OT2>
struct matrix_subtraction_traits<OT, vector<ET1, OT1>, vector<ET2, OT2>>
{
    using engine_type = matrix_subtraction_engine_t<OT, ET1, ET2>;
    using op_traits    = OT;
    using result_type  = vector<engine_type, op_traits>;

    static result_type subtract(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2)
};

template<class OT, class ET1, class OT1, class ET2, class OT2>
struct matrix_subtraction_traits<OT, matrix<ET1, OT1>, matrix<ET2, OT2>>
{
    using engine_type = matrix_subtraction_engine_t<OT, ET1, ET2>;
    using op_traits    = OT;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type subtract(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2)
};

template<class OT, class OP1, class OP2>
using matrix_subtraction_traits_t = ...;           //- Implementation-defined

```

§ 6.10.4. `matrix_multiplication_traits<OT, OP1, OP2>`

Class template `matrix_multiplication_traits<OT, OP1, OP2>` is an arithmetic traits type that performs the multiplication of two compatible *MathObjs* and returns the result in a *MathObj* having an implementation-defined engine type. There are eight partial specializations to support the eight binary multiplication operators described below.

Alias template `matrix_multiplication_traits_t<OT, OP1, OP2>` is used by the library to obtain the nested type `OT::multiplication_traits<OP1, OP2>`.

```

//- vector*scalar
//
template<class OT, class ET1, class OT1, class T2>
struct matrix_multiplication_traits<OT, vector<ET1, OT1>, T2>
{
    using scalar_type = detail::element_tag<T2>;
    using engine_type = matrix_multiplication_engine_t<OT, ET1, scalar_type>;
    using op_traits    = OT;
    using result_type  = vector<engine_type, op_traits>;

    static result_type multiply(vector<ET1, OT1> const& v1, T2 const& s2);
};

//- scalar*vector
//
template<class OT, class T1, class ET2, class OT2>
struct matrix_multiplication_traits<OT, T1, vector<ET2, OT2>>
{
    using scalar_type = detail::element_tag<T1>;
    using engine_type = matrix_multiplication_engine_t<OT, scalar_type, ET2>;
    using op_traits    = OT;
    using result_type  = vector<engine_type, op_traits>;

    static result_type multiply(T1 const& s1, vector<ET2, OT2> const& v2);
};

//- matrix*scalar
//
template<class OT, class ET1, class OT1, class T2>
struct matrix_multiplication_traits<OT, matrix<ET1, OT1>, T2>
{
    using scalar_type = detail::element_tag<T2>;
    using engine_type = matrix_multiplication_engine_t<OT, ET1, scalar_type>;
    using op_traits    = OT;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type multiply(matrix<ET1, OT1> const& m1, T2 const& s2);
};

//- scalar*matrix
//
template<class OT, class T1, class ET2, class OT2>
struct matrix_multiplication_traits<OT, T1, matrix<ET2, OT2>>

```

```

{
    using scalar_type = detail::element_tag<T1>;
    using engine_type = matrix_multiplication_engine_t<OT, scalar_type, ET2>;
    using op_traits    = OT;
    using result_type = matrix<engine_type, op_traits>;

    static result_type multiply(T1 const& s1, matrix<ET2, OT2> const& m2);
};

//- vector*vector
//
template<class OT, class ET1, class OT1, class ET2, class OT2>
struct matrix_multiplication_traits<OT, vector<ET1, OT1>, vector<ET2, OT2>>
{
    //- This specialization returns a scalar, and therefore does not compute
    //
    using op_traits    = OT;
    using elem_type_1 = typename vector<ET1, OT1>::element_type;
    using elem_type_2 = typename vector<ET2, OT2>::element_type;
    using result_type = matrix_multiplication_element_t<op_traits, elem_type_1, elem_type_2>;

    static result_type multiply(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2);
};

//- matrix*vector
//
template<class OT, class ET1, class OT1, class ET2, class OT2>
struct matrix_multiplication_traits<OT, matrix<ET1, OT1>, vector<ET2, OT2>>
{
    using engine_type = matrix_multiplication_engine_t<OT, ET1, ET2>;
    using op_traits    = OT;
    using result_type = vector<engine_type, op_traits>;

    static result_type multiply(matrix<ET1, OT1> const& m1, vector<ET2, OT2> const& v2);
};

//- vector*matrix
//
template<class OT, class ET1, class OT1, class ET2, class OT2>
struct matrix_multiplication_traits<OT, vector<ET1, OT1>, matrix<ET2, OT2>>
{
    using engine_type = matrix_multiplication_engine_t<OT, ET1, ET2>;
    using op_traits    = OT;

```

```

    using result_type = vector<engine_type, op_traits>;

    static result_type multiply(vector<ET1, OT1> const& m1, matrix<ET2, OT1> const& m2)
};

//- matrix*matrix
//
template<class OT, class ET1, class OT1, class ET2, class OT2>
struct matrix_multiplication_traits<OT, matrix<ET1, OT1>, matrix<ET2, OT2>>
{
    using engine_type = matrix_multiplication_engine_t<OT, ET1, ET2>;
    using op_traits    = OT;
    using result_type = matrix<engine_type, op_traits>;

    static result_type multiply(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2)
};

template<class OT, class OP1, class OP2>
using matrix_multiplication_traits_t = ...;    //- Implementation-defined

```

§ 6.11. Arithmetic operators

The arithmetic operators provide syntax that mimics common mathematical notation, with computation executed by an arithmetic traits type specified by one of the operands' operation traits template parameters.

Readers will note that the return types of the overloaded operators described below are left unspecified. This is a deliberate choice so that implementers have the freedom to choose whatever default technique for evaluating expressions they desire; for example, by returning temporary objects, or by using expression templates, or perhaps by some hybrid technique.

```

//- Negation
//
template<class ET1, class OT1>
inline auto
operator -(vector<ET1, OT1> const& v1)
{
    using op1_type    = vector<ET1, OT1>;
    using op_traits    = OT1;
    using neg_traits   = matrix_negation_traits_t<op_traits, op1_type>;

    return neg_traits::negate(v1);
}

template<class ET1, class OT1>
inline auto
operator -(matrix<ET1, OT1> const& m1)
{
    using op1_type    = matrix<ET1, OT1>;
    using op_traits    = OT1;
    using neg_traits   = matrix_negation_traits_t<op_traits, op1_type>;

    return neg_traits::negate(m1);
}

//- Addition
//
template<class ET1, class OT1, class ET2, class OT2>
inline auto
operator +(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2)
{
    using op_traits    = matrix_operation_traits_selector_t<OT1, OT2>;
    using op1_type     = vector<ET1, OT1>;
    using op2_type     = vector<ET2, OT2>;
    using add_traits    = matrix_addition_traits_t<op_traits, op1_type, op2_type>;

    return add_traits::add(v1, v2);
}

template<class ET1, class OT1, class ET2, class OT2>
inline auto
operator +(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2)
{
    using op_traits    = matrix_operation_traits_selector_t<OT1, OT2>;

```

```

        using op1_type    = matrix<ET1, OT1>;
        using op2_type    = matrix<ET2, OT2>;
        using add_traits  = matrix_addition_traits_t<op_traits, op1_type, op2_type>;

        return add_traits::add(m1, m2);
    }

// - Subtraction
//
template<class ET1, class OT1, class ET2, class OT2>
inline auto
operator -(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2)
{
    using op_traits      = matrix_operation_traits_selector_t<OT1, OT2>;
    using op1_type       = vector<ET1, OT1>;
    using op2_type       = vector<ET2, OT2>;
    using sub_traits     = matrix_subtraction_traits_t<op_traits, op1_type, op2_type>;

    return sub_traits::subtract(v1, v2);
}

template<class ET1, class OT1, class ET2, class OT2>
inline auto
operator -(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2)
{
    using op_traits      = matrix_operation_traits_selector_t<OT1, OT2>;
    using op1_type       = matrix<ET1, OT1>;
    using op2_type       = matrix<ET2, OT2>;
    using sub_traits     = matrix_subtraction_traits_t<op_traits, op1_type, op2_type>;

    return sub_traits::subtract(m1, m2);
}

// - Multiplication
// - vector*scalar and scalar*vector
//
template<class ET1, class OT1, class S2>
inline auto
operator *(vector<ET1, OT1> const& v1, S2 const& s2)
{
    static_assert(is_matrix_element_v<S2>);

    using op_traits      = OT1;

```

```

    using op1_type    = vector<ET1, OT1>;
    using op2_type    = S2;
    using mul_traits  = matrix_multiplication_traits_t<op_traits, op1_type,

    return mul_traits::multiply(v1, s2);
}

template<class S1, class ET2, class OT2>
inline auto
operator *(S1 const& s1, vector<ET2, OT2> const& v2)
{
    static_assert(is_matrix_element_v<S1>);

    using op_traits   = OT2;
    using op1_type    = S1;
    using op2_type    = vector<ET2, OT2>;
    using mul_traits  = matrix_multiplication_traits_t<op_traits, op1_type,

    return mul_traits::multiply(s1, v2);
}

//- matrix*scalar and scalar*matrix
//
template<class ET1, class OT1, class S2>
inline auto
operator *(matrix<ET1, OT1> const& m1, S2 const& s2)
{
    static_assert(is_matrix_element_v<S2>);

    using op_traits   = OT1;
    using op1_type    = matrix<ET1, OT1>;
    using op2_type    = S2;
    using mul_traits  = matrix_multiplication_traits_t<op_traits, op1_type,

    return mul_traits::multiply(m1, s2);
}

template<class S1, class ET2, class OT2>
inline auto
operator *(S1 const& s1, matrix<ET2, OT2> const& m2)
{
    static_assert(is_matrix_element_v<S1>);

```

```

        using op_traits    = OT2;
        using op1_type     = S1;
        using op2_type     = matrix<ET2, OT2>;
        using mul_traits   = matrix_multiplication_traits_t<op_traits, op1_type,

        return mul_traits::multiply(s1, m2);
    }

//- vector*vector
//
template<class ET1, class OT1, class ET2, class OT2>
inline auto
operator *(vector<ET1, OT1> const& v1, vector<ET2, OT2> const& v2)
{
    using op_traits    = matrix_operation_traits_selector_t<OT1, OT2>;
    using op1_type     = vector<ET1, OT1>;
    using op2_type     = vector<ET2, OT2>;
    using mul_traits   = matrix_multiplication_traits_t<op_traits, op1_type,

    return mul_traits::multiply(v1, v2);
}

//- matrix*vector
//
template<class ET1, class OT1, class ET2, class OT2>
inline auto
operator *(matrix<ET1, OT1> const& m1, vector<ET2, OT2> const& v2)
{
    using op_traits    = matrix_operation_traits_selector_t<OT1, OT2>;
    using op1_type     = matrix<ET1, OT1>;
    using op2_type     = vector<ET2, OT2>;
    using mul_traits   = matrix_multiplication_traits_t<op_traits, op1_type,

    return mul_traits::multiply(m1, v2);
}

//- vector*matrix
//
template<class ET1, class OT1, class ET2, class OT2>
inline auto
operator *(vector<ET1, OT1> const& v1, matrix<ET2, OT2> const& m2)
{
    using op_traits    = matrix_operation_traits_selector_t<OT1, OT2>;

```

```

    using op1_type    = vector<ET1, OT1>;
    using op2_type    = matrix<ET2, OT2>;
    using mul_traits  = matrix_multiplication_traits_t<op_traits, op1_type,

    return mul_traits::multiply(v1, m2);
}

//- matrix*matrix
//
template<class ET1, class OT1, class ET2, class OT2>
inline auto
operator *(matrix<ET1, OT1> const& m1, matrix<ET2, OT2> const& m2)
{
    using op_traits    = matrix_operation_traits_selector_t<OT1, OT2>;
    using op1_type     = matrix<ET1, OT1>;
    using op2_type     = matrix<ET2, OT2>;
    using mul_traits   = matrix_multiplication_traits_t<op_traits, op1_type,

    return mul_traits::multiply(m1, m2);
}

```

§ 7. Customization

The library provides for several forms of customization: custom element types, custom element promotion, custom engines, and custom arithmetical operations. The following sections show examples of each.

§ 7.1. Integrating a new element type

Suppose that you have created a new type that models a real number in some way and you wish for that type to be used as a matrix element:

```

class new_num
{
public:
    new_num();
    new_num(new_num&&) = default;
    new_num(new_num const&) = default;
    template<class U>    new_num(U other);

    new_num&    operator =(new_num&&) = default;
    new_num&    operator =(new_num const&) = default;
    template<class U>    new_num&    operator =(U rhs);

    new_num    operator -() const;
    new_num    operator +() const;

    new_num&    operator +=(new_num rhs);
    new_num&    operator -=(new_num rhs);
    new_num&    operator *=(new_num rhs);
    new_num&    operator /=(new_num rhs);

    template<class U>    new_num&    operator +=(U rhs);
    template<class U>    new_num&    operator -=(U rhs);
    template<class U>    new_num&    operator *=(U rhs);
    template<class U>    new_num&    operator /=(U rhs);

    ...
};

inline bool operator ==(NewNum lhs, NewNum rhs);
template<class U> inline bool operator ==(NewNum lhs, U rhs);
template<class U> inline bool operator ==(U lhs, NewNum rhs);
...
//- other comparison operators...
...
//- other arithmetic operators...
...
                inline new_num operator *(new_num lhs, new_num rhs);
template<class U> inline new_num operator *(new_num lhs, U rhs);
template<class U> inline new_num operator *(U lhs, new_num rhs);

```

Assuming that this type works as intended, and that all arithmetic interactions with other types are handled the set of operator overloads that you provide, then all that is required for the library to accept

`new_num` as an element type is to create a specialization of `number_traits`:

```
template<>
struct std::math::number_traits<new_num>
{
    using is_field      = true_type;
    using is_nc_ring    = true_type;
    using is_ring       = true_type;
};
```

By stating that `new_num` models at least a non-commutative ring, and by ensuring that its arithmetic operators functions as promised, the library's traits types will allow compilation to succeed.

§ 7.2. Custom element promotion

Suppose that you want the result of adding two `float` elements to be `double`. Then you would create the following custom types in your namespace:

```
//- Base template for custom element promotion
//
template<class T1, class T2>
struct element_add_traits_tst;

//- Promote any float/float addition to double.
//
template<>
struct element_add_traits_tst<float, float>
{
    using element_type = double;
};

//- Custom operation traits.
//
struct test_add_op_traits_tst
{
    template<class T1, class T2>
    using element_addition_traits = element_add_traits_tst<T1, T2>;
};
```

The new operation traits could be used like this:

```
matrix<fs_matrix_engine<float, 2, 3>, add_op_traits_tst>

matrix<dr_matrix_engine<float, allocator<float>>, add_op_traits_tst>

matrix<dr_matrix_engine<float, allocator<float>>, matrix_operation_traits>

//- mr1 --> matrix<fs_matrix_engine<double, 2, 3>, add_op_traits_tst>
//
auto mr1 = m1 + m1;

//- mr2 --> matrix<dr_matrix_engine<double, allocator<double>>, add_op_traits_tst>
//
auto mr2 = m1 + m2;

//- mr3 --> matrix<dr_matrix_engine<double, allocator<double>>, add_op_traits_tst>
//
auto mr3 = m1 + m3;
```

Note that this example assumes that an addition operation involving a fixed-size matrix and a dynamically-resizable matrix, or two dynamically-resizable matrices results in a dynamically-resizable matrix.

§ 7.3. Integrating a new engine type

Suppose that you want to add a custom fixed-size matrix engine that is somehow different from `fs_matrix_engine`; perhaps it is instrumented in some way for debugging, or uses fixed-size storage that is external to the engine object. It might look like this:

```

template<class T, int32_t R, int32_t C>
class fs_matrix_engine_tst
{
public:
    using engine_category = std::math::mutable_matrix_engine_tag;
    using element_type     = T;
    using value_type       = T;
    using reference         = T&;
    using pointer           = T*;
    using const_reference   = T const&;
    using const_pointer     = T const*;
    using difference_type   = std::ptrdiff_t;
    using index_type        = std::int_fast32_t;
    using size_type         = std::int_fast32_t;
    using size_tuple        = std::tuple<size_type, size_type>;

    using is_fixed_size    = std::true_type;
    using is_resizable     = std::false_type;

    using is_column_major  = std::false_type;
    using is_dense         = std::true_type;
    using is_rectangular   = std::true_type;
    using is_row_major     = std::true_type;

    using column_view_type = std::math::matrix_column_view<fs_matrix_eng
    using row_view_type    = std::math::matrix_row_view<fs_matrix_engin
    using transpose_view_type = std::math::matrix_transpose_view<fs_matrix_

public:
    constexpr fs_matrix_engine_tst();
    constexpr fs_matrix_engine_tst(fs_matrix_engine_tst&&) = default;
    constexpr fs_matrix_engine_tst(fs_matrix_engine_tst const&) = default;

    constexpr fs_matrix_engine_tst& operator =(fs_matrix_engine_tst&&)
    constexpr fs_matrix_engine_tst& operator =(fs_matrix_engine_tst cor

    constexpr const_reference operator ()(index_type i, index_type j) cor

    constexpr index_type columns() const noexcept;
    constexpr index_type rows() const noexcept;
    constexpr size_tuple size() const noexcept;

    constexpr size_type column_capacity() const noexcept;

```

```

constexpr size_type    row_capacity() const noexcept;
constexpr size_tuple   capacity() const noexcept;

constexpr reference    operator()(index_type i, index_type j);

constexpr void         assign(fs_matrix_engine_tst const& rhs);
template<class ET2>
constexpr void         assign(ET2 const& rhs);

constexpr void         swap(fs_matrix_engine_tst& rhs) noexcept;
constexpr void         swap_columns(index_type j1, index_type j2);
constexpr void         swap_rows(index_type i1, index_type i2);

private:
    ...                //- Implementation stuff
};

```

For each arithmetic operation in which you expect the new engine type to be involved, you will need to provide a specialization of the engine promotion traits for that operation. For example, let's assume that you're only interested in addition operations involving two operands having the new engine type, or where one operand has the standard fixed-size engine and the other has the new engine. Then your engine promotion traits might look like this:

```

//- Goal: Create a new fixed-size engine type and use it in arithmetic expr
//
template<class OT, class ET1, class ET2>
struct engine_add_traits_tst;

template<class OT, class T1, int32_t R1, int32_t C1, class T2, int32_t R2,
struct engine_add_traits_tst<OT,
    fs_matrix_engine_tst<T1, R1, C1>,
    fs_matrix_engine_tst<T2, R2, C2>>
{
    using element_type = std::math::matrix_addition_element_t<OT, T1, T2>;
    using engine_type   = fs_matrix_engine_tst<element_type, R1, C1>;
};

template<class OT, class T1, int32_t R1, int32_t C1, class T2, int32_t R2,
struct engine_add_traits_tst<OT,
    fs_matrix_engine_tst<T1, R1, C1>,
    std::math::fs_matrix_engine<T2, R2, C2>>
{
    using element_type = std::math::matrix_addition_element_t<OT, T1, T2>;
    using engine_type   = fs_matrix_engine_tst<element_type, R1, C1>;
};

template<class OT, class T1, int32_t R1, int32_t C1, class T2, int32_t R2,
struct engine_add_traits_tst<OT,
    std::math::fs_matrix_engine<T1, R1, C1>,
    fs_matrix_engine_tst<T2, R2, C2>>
{
    using element_type = std::math::matrix_addition_element_t<OT, T1, T2>;
    using engine_type   = fs_matrix_engine_tst<element_type, R1, C1>;
};

//- This is a custom operation traits type!
//
struct add_op_traits_tst
{
    template<class T1, class T2>
    using element_addition_traits = element_add_traits_tst<T1, T2>;

    template<class T1, class T2>

```

```
using engine_addition_traits = engine_add_traits_tst<T1, T2>;  
};
```

As we can see, these custom promotion traits dictate the resulting engine type for these particular cases. Resulting usage might look like this:

```
matrix<fs_matrix_engine<float, 2, 3>, matrix_operation_traits>  
  
matrix<fs_matrix_engine_tst<float, 2, 3>, add_op_traits_tst>  
  
matrix<dr_matrix_engine<float, allocator<float>>, matrix_operation_traits>  
  
//- mr1 --> matrix<fs_matrix_engine<float, 2, 3>, matrix_operation_traits>  
//  
auto    mr1 = m1 + m1;  
  
//- mr2 --> matrix<fs_matrix_engine_tst<double, 2, 3>, add_op_traits_tst>  
//  
auto    mr2 = m2 + m2;  
  
//- mr3 --> matrix<fs_matrix_engine_tst<double, 2, 3>, add_op_traits_tst>  
//  
auto    mr3 = m1 + m2;  
  
//- mr4 --> matrix<dr_matrix_engine<double, allocator<double>>, add_op_traits_tst>  
//  
auto    mr4 = m1 + m3;
```

Note that this example also assumes that an addition operation involving a fixed-size matrix and a dynamically-resizable matrix, or two dynamically-resizable matrices results in a dynamically-resizable matrix.

§ 7.4. Customizing an arithmetic operation

Suppose that you want to specialize the addition function for the addition of two matrices that employ the custom engine above and whose sizes happen to be 3x4.

```

//- Goal: Call a specialized addition function for addition of fixed-size n
// using the fixed-size test engine and having size 3x4.
//
template<class OT, class OP1, class OP2>
struct addition_traits_tst;

template<class OT>
struct addition_traits_tst<OT,
                        matrix<fs_matrix_engine_tst<double, 3, 4>, OT>,
                        matrix<fs_matrix_engine_tst<double, 3, 4>, OT>>
{
    using op_traits = OT;
    using engine_type = fs_matrix_engine_tst<double, 3, 4>;
    using result_type = matrix<engine_type, op_traits>;

    static result_type add(matrix<fs_matrix_engine_tst<double, 3, 4>, OT>
                          matrix<fs_matrix_engine_tst<double, 3, 4>, OT>);
};

//- This is a custom operation traits type!
//
struct test_add_op_traits_tst
{
    template<class T1, class T2>
    using element_addition_traits = element_add_traits_tst<T1, T2>;

    template<class OT, class ET1, class ET2>
    using engine_addition_traits = engine_add_traits_tst<OT, ET1, ET2>;

    template<class OT, class OP1, class OP2>
    using addition_traits = addition_traits_tst<OT, OP1, OP2>;
};

```

Actual usage might look like this:

```

matrix<fs_matrix_engine_tst<float, 3, 4>, add_op_traits_tst>    m1;
matrix<fs_matrix_engine_tst<double, 3, 4>, add_op_traits_tst>  m2;

//- mr1 --> matrix<fs_matrix_engine_tst<double, 3, 4>, add_op_traits_tst>
//
auto    mr1 = m1 + m1;    //- Calls matrix_addition_traits::add()

//- mr2 --> matrix<fs_matrix_engine_tst<double, 3, 4>, add_op_traits_tst>
//
auto    mr2 = m1 + m2;    //- Calls matrix_addition_traits::add()

//- mr3 --> matrix<fs_matrix_engine_tst<double, 3, 4>, add_op_traits_tst>
//
auto    mr3 = m2 + m2;    //- Calls matrix_addition_traits_tst::add()

```

§ 8. Meeting feedback

§ 8.1. Kona

At the Kona 2019 meeting, draft version D1 of this paper was reviewed by LEWG(I) and a joint session of SG14 and SG19. Both reviews were generally positive, several good suggestions were made, and some polls regarding future directions were taken.

§ 8.1.1. LEWG(I) Polls and Feedback (Wednesday 2019-02-20)

There were a few polls taken in this session:

1. We want 0-based indexing as opposed to 1-based indexing.
(unanimous: 20)
2. We like having separate row_vector and column_vector types in addition to matrix.
SF F N A SA (21 present)
3 0 5 4 4
3. We want explicitly named operations (e.g., dot and outer) in addition to operators.
SF F N A SA (21 present)
8 5 2 1 0

4. Define engine/matrix classes in terms of `mdspan` + storage and `mdspan` concepts (e.g., extents), and expose an `mdspan`-esque interface. This implies that `fs_` and `dyn_` are combined into one template parametrized on extents (which are either static or dynamic).

SF F N A SA (22 present)

6 2 7 0 0

There were some additional requests:

- Provide some implementation and usage experience.
- Provide a comparison with prior art.
- Explore the re-usability of `mdspan` and `common_type`.
- Be careful of allowing specializations of traits types that are part of namespace `std`; be consistent with other traits.

§ 8.1.2. Feedback From Joint SG14/SG19 Session (Friday 2019-02-22)

- Stick to 0-based indexing, for compatibility with current practice, and also for performance reasons.
- Provide a fixed-size engine whose memory is dynamically allocated.
- In this session, there was very broad agreement that the one-vector approach advocated by LEWG(I) was the way to proceed.
- Outer product computation is rare in practice, so, the vector-vector multiplication operator should return the inner product, and the outer product should be a named function.

§ 8.1.3. Other Suggestions Gathered at the Meeting

- Experiment with executors for concurrent operations.
- Include a "Gor table" (see [P1362R0](#), Section 4.4) showing feature levels and anticipated user sophistication for each.
- Include tutorial material on how the library can be used and extended, with several illustrative examples.